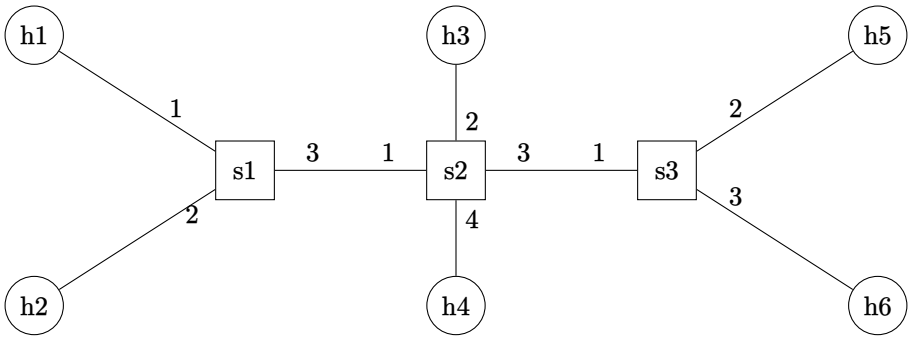
Software Defined Networking – 21 luglio 2025

Esercizio 1

Si consideri la rete in figura. I nodi h1,..., hn contengono tabelle ARP prepopolate per tutta la rete. Gli indirizzi MAC e IP dei nodi h1,..., hn sono MAC1,..., MACn e IP1,..., IPn. Sui nodi h1,..., hn sono in esecuzione processi in ascolto su tutte le porte TCP e UDP di interesse. Il controllore Ryu è collegato ai nodi s1,..., sm. La funzione di gestione dell'evento PacketIn è data nel Programma 1. Le interfacce tra gli switch e il controllore non sono indicate in figura. Se non altrimenti specificato, il TTL iniziale è 200. Negli switch sono preinstallate le regole in Tabella 1.



L'host h1 porta 3322 invia un datagramma UDP verso h5 porta 80.

1. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** Abbiamo un match con la seconda regola, quindi lo switch invia una packet-in al controllore. Il controllore risponde con un messaggio flow-mod.

L'host h1 porta 8080 invia un datagramma UDP verso h5 porta 80.

2. Scrivere se $s1$ invia messaggi al controllore. In caso affermativo scrivere quale tipo di messaggio openflow arriva in risposta dal controllore.

> **Soluzione:** Abbiamo un match con la nuova regola, quindi lo switch invia sull'uscita 3.

3. Scrivere se una qualche applicazione in $h5$ riceve il pacchetto. In caso affermativo, scrivere su quale socket è in ascolto tale applicazione. In caso negativo, scrivere in quale strato protocollare il pacchetto viene scartato.

> **Soluzione:** h5 riceve il pacchetto, che viene consegnato all'applicazione in ascolto sulla porta 443.

4. Quale regola il controllore può inserire in s3 per fare in modo che il pacchetto di risposta relativo a questa comunicazione –e soltanto ad essa– sia inoltrato all'host corretto senza passare dal controllore?

> **Soluzione:**
>
> ```
> priority = 20
> match = [ eth_type=IPv4, ip_dst=IP1, ip_src=IP5, ip_proto=UDP, udp_dst=8080, udp_src=443 ]
> action=[ SetField(udp_src=80), output(1) ]
> ```

Tabella 1: Regole iniziali

Switch s1		
Priority	*Match*	*Action*
5	*	DecTTL, Output(FLOOD)
10	eth_type = IPv4, ip_proto=UDP	Output(CONTROLLER)
15	eth_dst = MAC2	Output(2)
15	eth_dst = MAC1	Output(1)
Switch s2		
Priority	*Match*	*Action*
0	*	DecTTL, Output(FLOOD)
10	eth_type = IPv4, ip_proto = UDP	Output(FLOOD)
10	eth_type = IPv4, ip_proto = TCP	Output(CONTROLLER)
Switch s3		
Priority	*Match*	*Action*
0	*	DecTTL, Output(FLOOD)
15	eth_dst = MAC5	Output(2)
15	eth_dst = MAC6	Output(3)

Programma 1 – Packet In

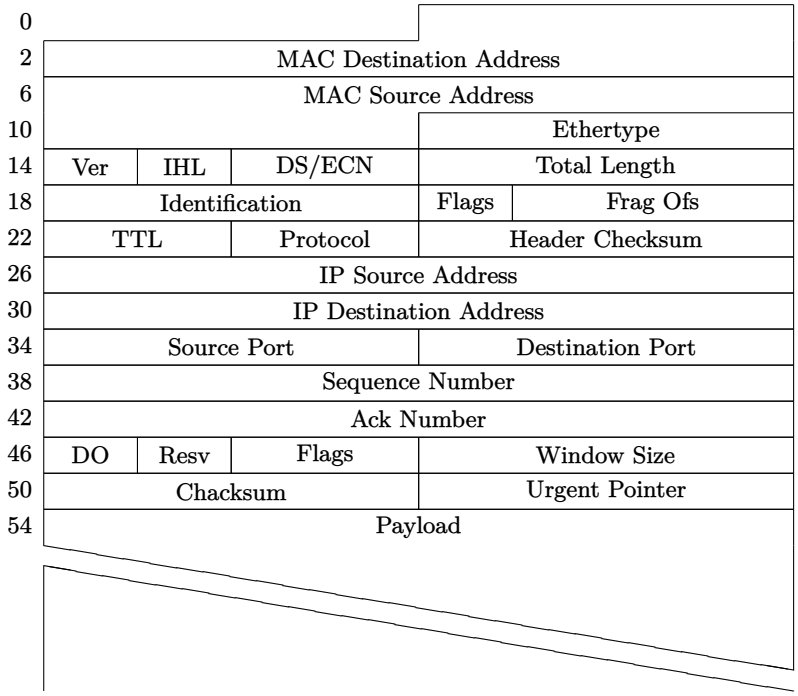
```
@set_ev_cls(ofp_event.EventOFPPacketIn, MAIN_DISPATCHER)
def packet_in_handler(self, ev):
   msg = ev.msg; datapath = msg.datapath; ofproto = datapath.ofproto;
   parser = datapath.ofproto_parser; in_port = msg.match['in_port']
   pkt = packet.Packet(msg.data); pkt_eth = pkt.get_protocol(eth.eth)
   pkt_ipv4 = pkt.get_protocol(ipv4.ipv4)
   pkt_udp = pkt.get_protocol(udp.udp); pkt_tcp = pkt.get_protocol(tcp.tcp)
   data = msg.data

   if (datapath.id == 1 and pkt_udp is not None and udp_dst == 80):
      match = parser.OFPMatch(eth_type=ETH_TYPE_IP,
         ipv4_dst = pkt_ipv4.dst, ip_proto=IPPROTO_UDP,
         udp_dst = pkt_udp.dst_port)
      actions = [
         parser.OFPActionSetField(udp_dst = 443)
         parser.OFPActionOutput(3) ]
      inst = [ parser.OFPInstructionActions(
         ofproto.OFPIT_APPLY_ACTIONS,actions)]
      mod = parser.OFPFlowMod(datapath=datapath, priority=20,
         match=match, instructions=inst)
      datapath.send_msg(mod)
      return

   actions = [parser.OFPActionOutput(ofproto.OFPP_FLOOD)]
   out = parser.OFPPacketOut(datapath=datapath,
            buffer_id=ofproto.OFP_NO_BUFFER,data=msg.data,
            in_port=in_port, actions=actions)
   datapath.send_msg(out)
```

Esercizio 2

Pacchetto Ethernet/IP/UDP



Si consideri il seguente filtro BPF applicato a un pacchetto Ethernet/IP/TCP.

```
0 (000) ldh [12]
1 (001) jeq #ETHERTYPE_IP jt 2 jf 9
2 (002) ldb [23]
3 (003) jeq #IPPROTO_TCP jt 4 jf 9
4 (004) ldh [34]
5 (005) jeq #123 jt 8 jf 6
6 (006) ldh [36]
7 (007) jeq #123 jt 8 jf 9
8 (008) ret #262144
9 (009) ret #0
```

5. Quali righe sono eseguite per decidere se catturare un pacchetto TCP diretto alla porta 123?

   Soluzione: 0, 1, 2, 3, 4, 5, 6, 7, 8

   Nota: Potrebbe terminare prima se anche la porta sorgente fosse 123.

6. Quali righe sono eseguite per decidere se catturare un pacchetto UDP diretto alla porta 123?

   Soluzione: 0, 1, 2, 3, 9

7. Qual è il massimo numero di istruzioni eseguite? Com'è fatto il pacchetto che causa l'esecuzione di questo numero di istruzioni?

Soluzione: 9 istruzioni (0, 1, 2, 3, 4, 5, 6, 7, 8 o 9)

Si tratta di un pacchetto TCP con porta sorgente diversa da 123.

8. Si vuole modificare il filtro in modo da catturare soltanto i primi 50 byte del pacchetto. Quale riga deve essere modificata e come?

Soluzione: La riga da modificare è la 8. Si deve cambiare il valore restituito in modo da considerare solo i primi 50 byte del pacchetto.

```
(008) ret #50
```

Esercizio 3

Un ateneo fornisce un servizio di WiFi gratuito per gli studenti. Per evitare che gli studenti possano abusare del servizio, il sistema di monitoraggio tiene traccia degli indirizzi MAC di ciascuno. Si assuma che ogni studente usi sempre lo stesso MAC.

I pacchetti ricevuti da ogni studente sono conteggiati usando un Count-Min sketch A con k funzioni hash, ciascuna delle quali con output di b bit. Gli utenti che nel mese corrente hanno ricevuto più di $S = 5\,\text{GB}$ di traffico ricevono un servizio limitato. *Usare i prefissi decimali del sistema internazionale.*

Si consideri il caso con $k = 5$ funzioni hash, ciascuna delle quali con un output di $b = 12$ bit. Si considerno pacchetti tutti di dimensione $1\,\text{kB}$. Alla data odierna si sono collegati alla rete $n = 10\,000$ studenti, 100 dei quali sono già superato la soglia S mentra gli altri hanno ricevuto pochissimi pacchetti.

Sia x il MAC x di uno studente, $\lambda(x)$ i pacchetti ricevuti.

9. Con riferimento alla prima funzione hash, qual è la probabilità che uno studente che si collega per la prima volta sia conteggiato nello stesso elemento di un altro studente già osservato?

Soluzione: La probabilità che il MAC di almeno uno degli altri studenti collida con il MAC del nuovo studente è

$$p = 1 - \left(1 - \frac{1}{m}\right)^n = 1 - \exp\left(-\frac{n}{2^b}\right) = 91\%$$

10. Qual è la probabilità che uno studente che si collega per la prima volta riceva un servizio limitato?

Soluzione: La probabilità che il MAC di almeno uno degli altri studenti che hanno superato la soglia collida con il MAC del nuovo studente è

$$p = 1 - \exp\left(-\frac{100}{2^b}\right) = 2{,}4\%$$

La probabilità di collisione su tutti gli hash è

$$p^k = 7{,}9 \cdot 10^{-9}$$

11. Qual è la probabilità che uno studente che ha superato la soglia riceva un servizio limitato?

Soluzione: 1, perché un CM sketch non ha falsi negativi

12. A una certa data, gli utenti del servizio hanno ricevuto globalmente $\Lambda = 600\,\text{GB}$ byte di traffico. Quanto vale la somma di ognuna delle righe del Count-Min sketch?

Soluzione: La somma di ognuna delle righe del Count-Min sketch è pari a

$$\frac{\Lambda}{1\,\text{kB}} = 600 \cdot 10^6$$

Esercizio 4

Si consideri il seguente set di regole, per il quale implementare un meccanismo di Bit Vector Linear Search:

Indirizzo MAC Destinazione	Indirizzo IP Destinazione
02:1A:3F:8B:C7:D4	44.0.0.0/6
F6:5C:9E:2D:1B:0A	48.0.0.0/5
02:1A:3F:8B:C7:D4	32.0.0.0/5
4E:70:A3:6F:C1:89	96.0.0.0/3
4E:70:A3:6F:C1:89	64.0.0.0/3
B2:D5:E8:21:74:9C	*

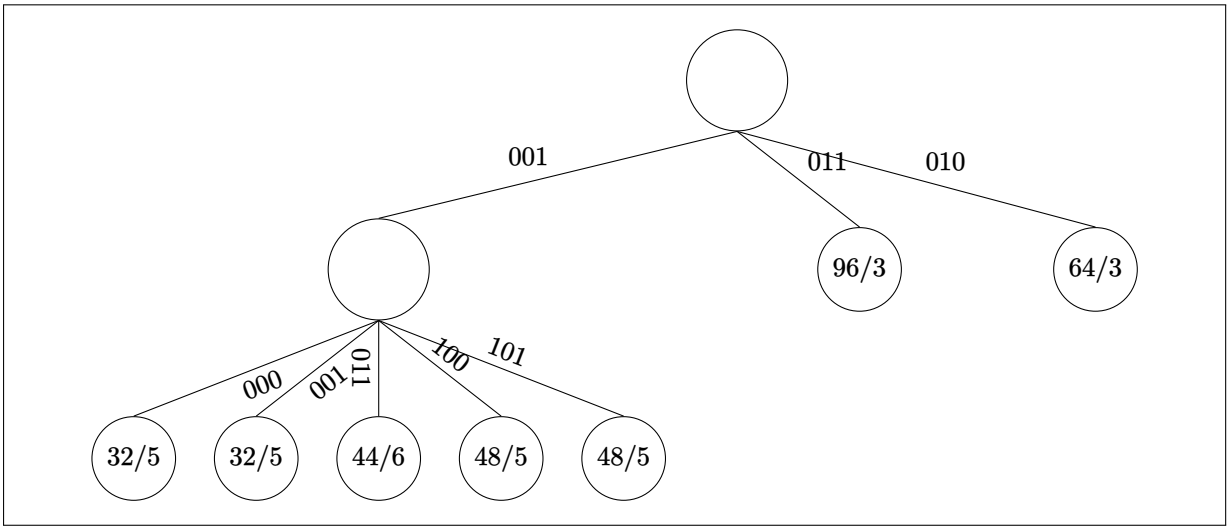
13. Definire il bitvector set per il campo IP destinazione.

Soluzione:

Indirizzo IP Destinazione	Bitvector
32.0.0.0/5	001001
44.0.0.0/6	100001
48.0.0.0/5	010001
64.0.0.0/3	000011
96.0.0.0/3	000101
*	000001

14. L'algoritmo di ricerca del prefisso IP destinazione è implementato usando un multibit trie a passo 3. Disegnare il trie.

Soluzione:



15. Definire il bitvector set per il campo MAC destinazione.

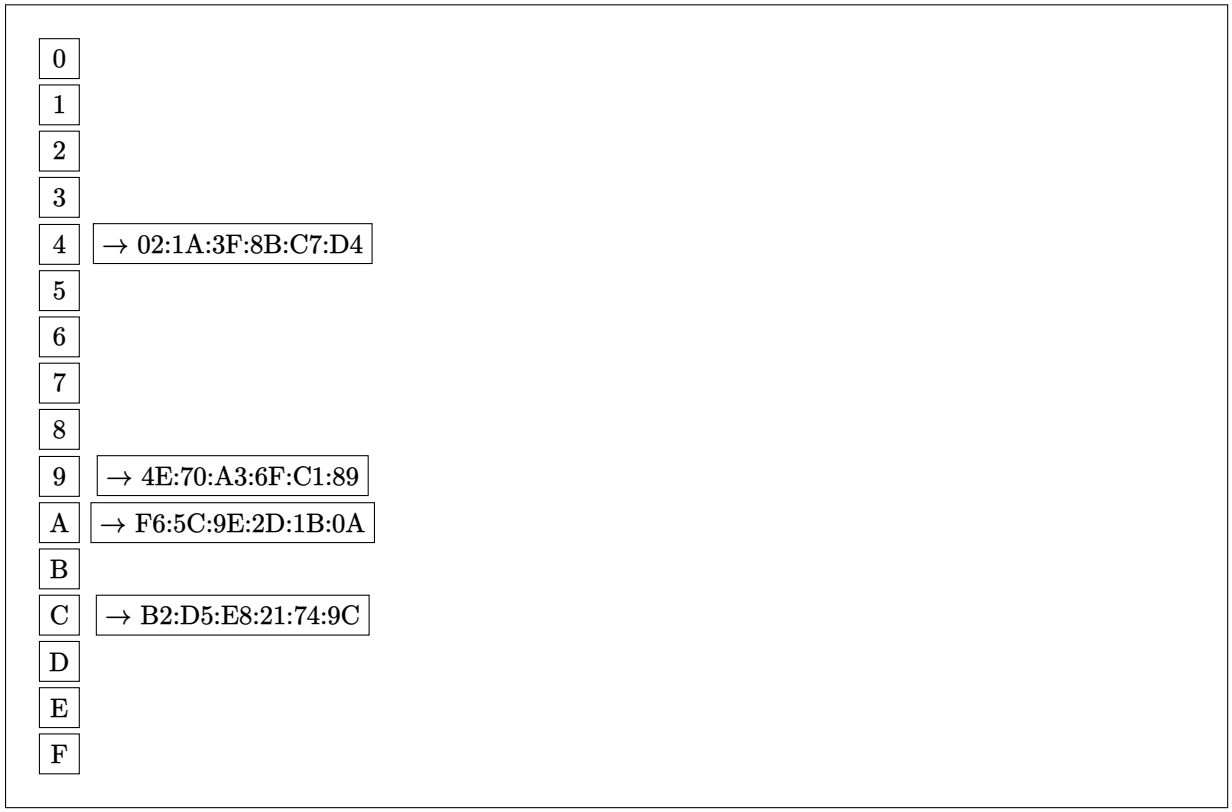
Soluzione:

Indirizzo MAC Destinazione	Bitvector
02:1A:3F:8B:C7:D4	101000
F6:5C:9E:2D:1B:0A	010000
4E:70:A3:6F:C1:89	000110
B2:D5:E8:21:74:9C	000001

16. L'algoritmo di ricerca del prefisso MAC destinazione è implementato usando una hash table definita dalla funzione $h(x) = x \bmod 16$. Disegnare la tabella.

Soluzione:

.


0
1
2
3
4
→ 02:1A:3F:8B:C7:D4
5
6
7
8
9
→ 4E:70:A3:6F:C1:89
A
→ F6:5C:9E:2D:1B:0A
B
C
→ B2:D5:E8:21:74:9C
D
E
F